Web Systems Development

Security: is it a feature you add, or a property of how the system was designed?

Session 19 · Tue 11/3 · the OWASP Top 10, and six holes in one to-do app

RPI

AGENDA

- What is role of security?
- Common attack vectors
- How frameworks protect you
- Drills

PROJECT MIDTERM

Team scores were high. Final individual grades will incorporate contribution and peer evaluations.

Lead

13 of 56 students

Most own commits on the team (ties broken by lines added)

Keep merging by pull request with a review. Split HW7 so every name lands in the history.

Solid

25 of 56 students

Everyone not in the other three tiers

Same pattern through HW7: your branch, a pull request, at least one review.

Thin

9 of 56 students

1 commit under 200 lines, or < 40% of team average commits and lines

Nothing reduced now. Reduced at the final if the history still looks like this.

None

9 of 56 students

0 commits of your own (merges do not count)

Final project grade will be reduced below the team's unless your commits show substantial work from here.

The function. tier = f(your own commits, your lines added, your team's averages), from every branch of the team repo, matched to you by RPI email or GitHub account. Merging a teammate's pull request credits their work, not yours. Lockfiles, images and vendor/ never count as lines. Your Blackboard feedback has your numbers and your rank on the team and in the class.

READING · WEEK 11

Five required readings prepare Friday's security lab

REQUIRED · 1–5

1. **OWASP · Top 10:2025**
   Introduction; A01, A02, A03, A05
2. **OWASP · Cross Site Scripting Prevention Cheat Sheet**
   Introduction through Output Encoding — HTML, attribute, JavaScript
3. **OWASP · CSRF Prevention Cheat Sheet**
   Introduction; Synchronizer Token Pattern; SameSite
4. **Laravel docs · Blade; Configuration; Sanctum**
   Displaying Data; Environment Configuration; How it Works
5. **MDN · Practical security implementation guides**
   CSP; Clickjacking; MIME types; TLS — HSTS section

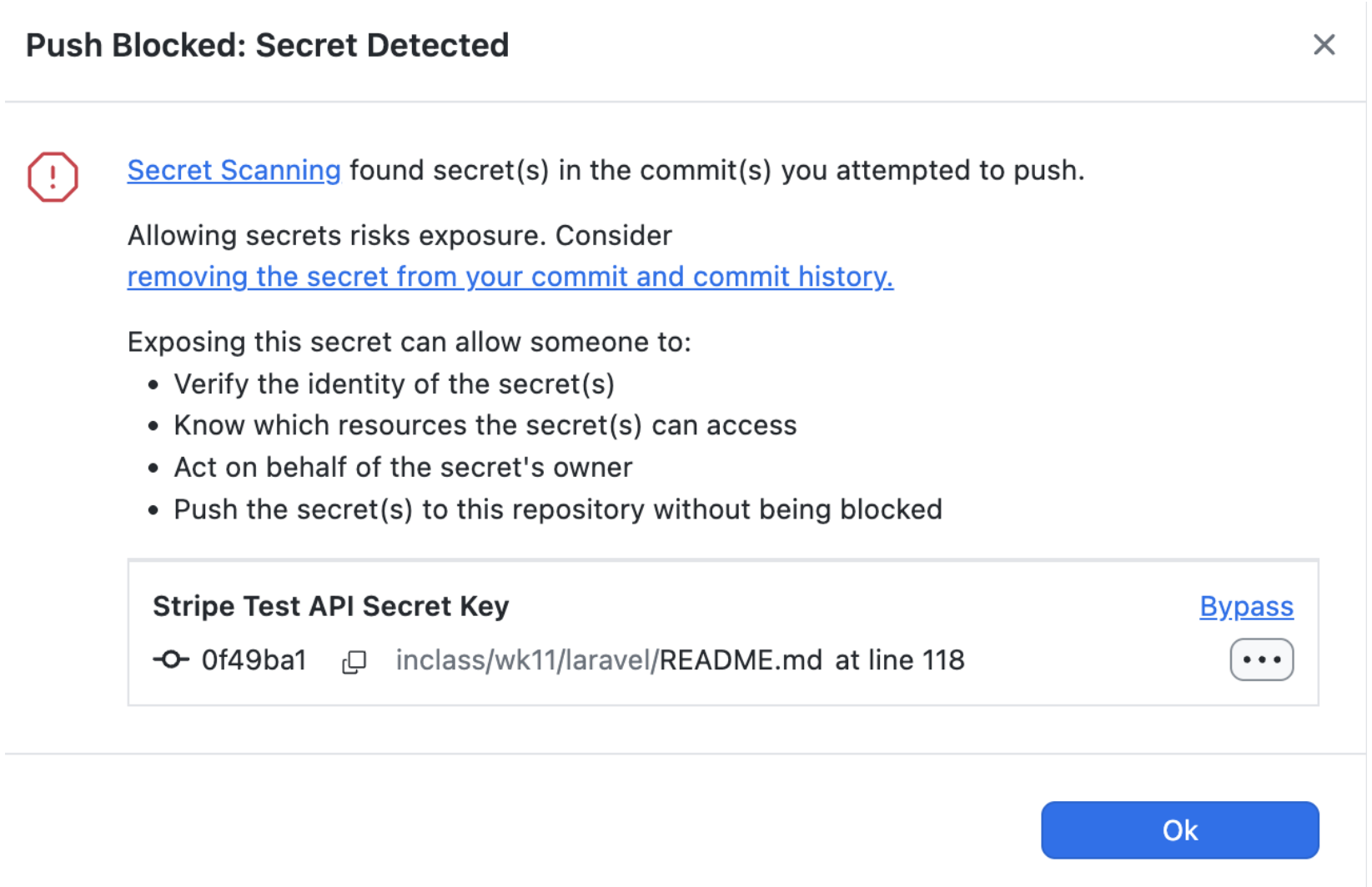
OPTIONAL · 6–9

6. **GitHub · a more secure npm supply chain**
   Also: composer audit; npm audit
7. **Saltzer & Schroeder · Protection of Information (1975)**
   §I.A.3 · Design Principles
8. **Wikipedia · XZ Utils backdoor**
   Background; Mechanism
9. **McDonald · Web Security for Developers**
   RPI library · chapters 6, 7, 8

DUE THIS WEEK

Homework 7 — your team's Laravel back end
Fri 11/6 · 11:59 PM

In-class — push inclass/wk11/laravel
Fri 11/6 · end of class



SECURITY CAN BE DESIGNED INTO PRODUCTS.

Top 10:2025 List

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. A05:2025 - Injection
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures
9. A09:2025 - Security Logging and Alerting Failures
10. A10:2025 - Mishandling of Exceptional Conditions

BUILDS ON

Where this sits: code you have already written

YOU ALREADY DID

week 3

prepare() and ? placeholders in PHP

week 7

Blade: {{ }} escapes by default; @csrf

week 9

Eloquent: where() binds the value

HW 7

your team's api/: tables, models, five routes

TODAY

each of those

seen as the attack it was preventing

plus three

you have not met: secrets, dependencies, headers

the map

OWASP's ten names for what goes wrong

one idea

assume every input is hostile; fail closed

NEXT

Fri 11/6

Group meetings on the backend

HW 8

the same six on your own api/

Tue 11/10

authentication: logging in, sessions vs. tokens, SSO

HW 9

the login and the write endpoints

You have been secure all term — by default

- Blade escaped your output. Eloquent bound your queries. .env stayed out of git. Laravel checks a CSRF token on every POST.
- You never saw any of it, because the defaults are the security. Today we turn them off, and watch.



What the framework already does for you

OUTPUT

`{{ $x }}`

Blade escapes every value, so a <script> in your data shows as text. (week 7, drill 4)

QUERIES

`where('a', $v)`

Eloquent sends the value bound, separate from the SQL, so it can't be injected. (week 9; week 3's prepare())

FORMS

`@csrf`

Every POST must carry a token Laravel checks, so another site can't post as you. (this week's reading; your project's api.js does it by hand)

SECRETS

`.env`

.gitignore keeps your keys out of the repository without your doing anything. (every Laravel app since week 7)

OWASP Top 10

The field's shared list of what goes wrong

A01	Broken Access Control	drill 3 · next week	A02	Security Misconfiguration	drills 4, 6
A03	Software Supply Chain Failures	drill 5	A04	Cryptographic Failures	today
A05	Injection	drills 1, 2	A06	Insecure Design	today
A07	Authentication Failures	today · next week	A08	Software or Data Integrity Failures	today
A09	Security Logging & Alerting Failures	today	A10	Mishandling of Exceptional Conditions	today

4. Cryptographic Failures

MORE NEXT WEEK WHEN WE DEAL WITH USER PASSWORD

Data that needed protecting was stored or sent without it, or with cryptography done wrong.

WHAT IT IS

Passwords stored in plain text or with a fast hash; tokens that are guessable; traffic over HTTP; a hard-coded key.

The failure is usually not a broken algorithm. It is the right one not used, or the key handled badly.

Fell from 2nd to 4th as frameworks made the right thing the default.

EXAMPLES

`md5($password)` in the users table: a leaked table is cracked in hours with a rainbow table.

The session cookie sent over plain HTTP on the café wifi; whoever reads it is you.

A password-reset token built from the user id and the time.

The password hash included in the JSON for GET /api/users (drill 4's cousin).

HOW LARAVEL HELPS

`'password' => 'hashed'` cast: bcrypt with a per-user salt and a cost (BCRYPT_ROUNDS); Hash::check compares.

`Crypt::encrypt` AES-256 with APP_KEY from key:generate; cookies and sessions are encrypted this way by default.

`Str::random(60)` for tokens: random_bytes underneath, never time or ids.

`#[Hidden(['password'])]` keeps the hash out of every JSON response; SESSION_SECURE_COOKIE and HSTS keep the cookie on HTTPS.

6. Insecure Design

Every line is correct and the system is still unsafe, because the plan had the hole.

WHAT IT IS

Missing or wrong decisions made before coding: what must be checked, what can be abused, what happens under failure.

Business logic flaws: a step that can be skipped, a limit that is not enforced, a value trusted from the client.

Cannot be fixed by a patch; it needs a design change. The cure is threat modelling: who could misuse this, and how?

EXAMPLES

A checkout where the browser sends the price, and the server charges that.

A coupon code with no limit on how many times one account uses it.

Password reset by "what was your first pet", answerable from social media.

A midterm prototype whose rules.js decides who may delete, with no server check behind it.

HOW LARAVEL HELPS

No framework can design your app. What Laravel gives is a place to put each decision once:

`validate()` rules say what a request may contain; the price comes from the database, not the body.

`Policy` classes say who may do what, in one file per model, read by every route.

`RateLimiter::for()` and queued jobs for the limits and the slow paths; php artisan test to pin the rules down.

7. Authentication Failures

MORE NEXT WEEK WHEN WE DEAL WITH USER AUTHENTICATION

Confirming who someone is, done badly: weak passwords, unlimited guessing, sessions that can be stolen or fixed.

WHAT IT IS

Credential stuffing (passwords leaked elsewhere), brute force, weak or default passwords, missing multi-factor.

Session problems: an id that is not regenerated at login, never expires, or is exposed in a URL.

Error messages that say which half of the login was wrong.

EXAMPLES

A login route with no limit: a script tries 10,000 passwords a minute.

"No account with that email" versus "Wrong password": now the attacker knows which emails exist.

The session id stays the same after login, so an id planted before login is yours after (session fixation).

Log out that only deletes the cookie in the browser; the server session lives on.

HOW LARAVEL HELPS

`Two-factor authentication (2FA)` via Fortify adds a second proof beyond the password, using authenticator codes and recovery codes.

`Auth::attempt()` hashes and compares; one generic message for both failures.

`session()->regenerate()` after login; invalidate() and regenerateToken() at logout.

`throttle:login` RateLimiter keyed by email and IP blocks repeated guessing.

`Strong passwords + secure cookies` reduce credential and session risk.

https://laravel.com/framework/docs/12.x/fortify#two-factor-authentication

8. Software or Data Integrity Failures

Code or data is trusted without checking that it is what it claims to be, or that nobody changed it on the way.

WHAT IT IS

Updates installed without a signature, scripts loaded from a CDN with no integrity check, a build pipeline anyone can edit.

Insecure deserialization: the client sends an object, the server rebuilds it and runs what is inside.

The difference from A03: this is about verifying; A03 is about where the code came from.

EXAMPLES

`unserialize($_COOKIE['cart'])` rebuilds whatever object the visitor wrote into the cookie.

`<script src="https://cdn…/lib.js">` with no integrity= attribute: if the CDN is compromised, so is every visitor.

A CI job that curls a script from a URL and runs it on every build.

HOW LARAVEL HELPS

`EncryptCookies` every cookie, and the session, is encrypted and signed with APP_KEY; a changed byte is rejected.

`URL::signedRoute()` and the signed middleware: a link whose parameters cannot be edited (unsubscribe, email verification).

`SerializesModels` queued jobs store a model's id and reload it, never a serialized object from outside.

`composer.lock` records a hash for every package, so what is installed is what was locked.

9. Security Logging and Alerting Failures

The attack happened and nobody noticed, because nothing was logged, or nothing read the log.

WHAT IT IS

Logins, failures, permission denials and high-value actions not recorded; logs kept only on the one machine; nobody alerted.

Breaches are typically found months later, by someone else. Logging is what makes the time shorter.

The opposite failure too: logging passwords, tokens or card numbers.

EXAMPLES

10,000 failed logins overnight, and the first anyone hears of it is a customer.

A flood of 500 errors from an injection probe that nobody looked at (drill 1, from the other side).

`Log::info($request->all())` writes the password field into the log file.

The container restarts and the log, which lived inside it, is gone.

HOW LARAVEL HELPS

`Log::warning()` and channels in config/logging.php: stack, daily files, Slack, a log service, all from one call.

`Illuminate\Auth\Events\Failed` Login, Logout and Lockout events you can listen to and count.

`report()` and the exception handler in bootstrap/app.php send every unhandled error to the log with context.

`Log::withContext()` tags every line of a request with the user and request id; LOG_LEVEL per environment.

10. Mishandling of Exceptional Conditions

What the app does when something goes wrong: fails open, leaks details, or leaves data half-written.

WHAT IT IS

New in 2025. Errors, timeouts, nulls and partial failures handled in a way an attacker can trigger or exploit.

Fail open: a check throws, the exception is swallowed, the request continues as if it passed.

Inconsistent state: step one wrote to the database, step two failed, nothing undid step one.

EXAMPLES

`catch (Exception $e) {}` around the authorization check: when it throws, the delete goes ahead.

A payment that fails after the order row was inserted; the customer has an order and no charge.

The error response is the raw exception as JSON, with the query and the file path in it.

The rate-limit service is down, so the login route allows everything until it is back.

HOW LARAVEL HELPS

`withExceptions` in bootstrap/app.php: every exception becomes a consistent response (404, 403, 422, 500) with no internals in production.

`DB::transaction(fn () => …)` rolls back every write in the closure if any step throws.

`validate()` fails closed: a bad request is a 422 before the controller runs; findOrFail and abort() the same.

`retry()` queued jobs with tries and failed(); nothing silently continues.

SQL Injection

Drill



5. SQL Injection

Text a stranger typed is read as code: SQL by the database, HTML and JavaScript by the browser, a command by the shell.

WHAT IT IS

Any place your program builds a command by pasting in a string is a place the string can change the command.

SQL injection, cross-site scripting (XSS), OS command injection, LDAP and template injection are one bug in different languages.

Fell from 3rd to 5th because ORMs and template engines escape by default. It still appears everywhere those defaults are turned off.

EXAMPLES

`' OR 1=1 --` in a search box returns every row (drill 1).

`' UNION SELECT id, email, 0 FROM users --` returns the users table through the to-do list.

`<script>alert(1)</script>` in a to-do note runs in every other user's browser (drill 2).

`orderBy($request->query('sort'))` lets a visitor name any column, or an expression.

HOW LARAVEL HELPS

`Eloquent` is Laravel's ORM: you work with models such as Todo, and it translates methods like where() into SQL.

`where('title', 'like', $q)` creates SQL with a placeholder, such as WHERE title LIKE ?, and sends $q separately.

`"Bind every value"` means user data stays data—it cannot become SQL syntax. Do not concatenate input or use raw SQL; column names and sort directions must be allow-listed.

What does this search box return?

PREDICT The to-do search, as the lab's starter app has it. A visitor types the second line into the box.

```
$sql = "select id, title, done from todos where title like '%$q%' order by title";

the search box:    ' UNION SELECT id, email, 0 FROM users WHERE email LIKE '%
```

Predict with a partner: write out the SQL the database actually receives. What table does it read? (Hint: the typed quote closes the like-string, and UNION glues a second query onto the first.)

A search box that reads the users table

A05:2025 Injection *drill 1 · TodoController::index(), the to-do search from HW6/HW7*

```
public function index(Request $request) {
    $q = trim((string) $request->query('q', ''));
    $sql = "select id, title, done from todos
            where title like '%$q%' order by title";
    $todos = collect(DB::select($sql));
    $todos = Todo::where('title', 'like', "%$q%")
                 ->orderBy('title')->get();
    return view('todos.index', ['todos' => $todos, 'q' => $q]);
}
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

A visitor types ' UNION SELECT id, email, 0 FROM users WHERE email LIKE '% into the search box. The list fills with every user's email. The same trick reaches password hashes.

WHY THE FIX WORKS

The typed text was pasted into the SQL, so it became SQL. A bound parameter (?) sends the value separately — the database treats it only as text to compare, never as a command. where() does this for you.

THE FIX, IN ONE LINE

Todo::where('title','like',"%$q%")

XSS Injection

Drill



XSS makes untrusted text execute in the browser

Cross-site scripting (XSS) *a browser-side injection vulnerability*

```
untrusted input:
<script>document.body.className='hacked';</script>

unsafe page output:
<div class="note">{!! $note !!}</div>

browser result: page content + attacker code
```

Key idea: the application delivers the payload; the victim's browser executes it.

WHAT IT IS

Cross-site scripting occurs when untrusted data is inserted into a page without the correct output encoding or sanitization. The browser treats attacker-controlled characters as HTML or JavaScript instead of plain text.

WHAT IT CAN DO

XSS can read or change page content, send requests as the signed-in user, capture data entered on the page, and transmit information to an attacker.

CORE DEFENSES

Encode output for its context; avoid raw HTML; sanitize allowed markup; use Content Security Policy as defense in depth.

A note that runs in everyone's browser

A05:2025 Injection (XSS) *drill 2 · todos/show.blade.php, a note on a shared to-do*

```
{{-- resources/views/todos/show.blade.php --}}
<h2>Notes</h2>
~~<div class="notes">{!! $todo->notes ?: 'No notes.' !!}</div>~~
<div class="notes">{{ $todo->notes ?: 'No notes.' }}</div>

the note someone left on the shared "Pantry potluck" to-do:
  <script>alert("I run in your browser now. Your cookies: "
    + document.cookie + ". I can read this page, post to your"
    + " list as you, and send it all to my server.")</script>
  Sign up here!
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

A stranger typed a <script> into a note. {!! !!} prints it as HTML, so your browser runs it inside your page, as you: it can read the page, post with your session, and send it all to a server it names. HttpOnly keeps the session cookie out of the list; the script never needed it.

WHY THE FIX WORKS

{{ }} escapes first: < becomes <, so the browser shows the characters instead of running them. Blade escapes by default; the bug is the one place that asked it not to.

THE FIX, IN ONE LINE

{{ $todo->notes }} not {!! … !!}

Injection, in two lines

A05 · DATABASE · DRILL 1

SQL injection

Never build SQL by pasting in input. Bind the value: where('title','like',$q) sends it separately.

A05 · BROWSER · DRILL 2

Cross-site scripting

Never print untrusted data as raw HTML. {{ }} escapes; {!! !!} does not.

The idea under all of them: the attacker can send anything — so never let what they send be read as code. Bind it, or escape it.

Cross-site request forgery (CSRF)

A01:2025. Your browser attaches your cookie to a request to a site — even when another site is what triggered the request.

Drill 3 on Friday. Your project's React api.js carries the same defense by hand.

CSRF tricks your browser into acting with your identity

1 · THE SETUP

You are already signed in

The browser stores a session cookie for the target site. That cookie identifies the signed-in account—it does not prove the user intended every request.

2 · THE FORGERY

Another site triggers the action

An attacker's page submits a request to the target. If the browser includes your cookie and the server lacks a CSRF check, the action runs as you.

Key distinction: CSRF forges an authenticated action; it does not need to steal the password or cookie. Tokens, SameSite cookies, and Origin or Sec-Fetch checks help reject the forgery.

1. Broken Access Control

A user does something the rules say they may not: read, change or delete what is not theirs, or act as an admin.

WHAT IT IS

Access control is the rule "who may do what to which thing". It is broken when the server does not check the rule on every request.

The page hiding a button is not access control. The request behind the button is what matters.

In 2025 this entry also holds CSRF (a request your browser sends for another site) and SSRF (your server fetching a URL an attacker chose).

EXAMPLES

`PUT /api/reviews/17` succeeds for any logged-in user, not just the owner of 17.

Change the id in the URL from your record to someone else's and the page shows it.

`/admin` is not linked anywhere, so nobody protected it.

Another site's form posts to your /todos and your browser attaches your cookie (CSRF, drill 3).

HOW LARAVEL HELPS

`auth:sanctum` middleware on the route group: no login, no request reaches the controller (401).

`Gate::authorize('update', $review)` asks the policy; a false becomes a 403 before any code runs.

`$request->user()->reviews()` scopes the query to the login, so another user's id finds nothing (404).

`@csrf` and the X-XSRF-TOKEN header: every POST must carry a token the other site cannot read; SameSite cookies as the second lock.

A page on another site. Does the server accept it?

PREDICT You use the to-do app in one tab. In another tab you open a page on a different site, which contains only this:

```
<form id="evil" method="POST" action="http://localhost:8000/todos">
    <input type="hidden" name="title" value="Call the attacker: 555-0100">
</form>
<script>document.getElementById('evil').submit();</script>
```

Predict: the server receives a POST it did not ask for, from a page it did not serve. Can it tell? Does it add the to-do? (Hint: what, in the request itself, says which page sent it — and what does a cookie actually prove?)

Anytime you define a "POST", "PUT", "PATCH", or "DELETE" HTML form in your application, you should include a hidden CSRF `_token` field in the form so that the CSRF protection middleware can validate the request. For convenience, you may use the `@csrf` Blade directive to generate the hidden token input field:

```
<form method="POST" action="/profile">
    @csrf

    <!-- Equivalent to... -->
    <input type="hidden" name="_token" value="{{ csrf_token() }}"
</form>
```

Another site adds to-dos to your list

A01:2025 Broken Access Control *drill 3 · todos/index.blade.php and bootstrap/app.php*

```
{{-- resources/views/todos/index.blade.php --}}
<form method="POST" action="{{ route('todos.store') }}" class="add">
    @csrf
    <input type="text" name="title" placeholder="New to-do" required>
    <button type="submit">Add</button>
</form>

bootstrap/app.php
~~$middleware->validateCsrfTokens(except: ['todos']);~~
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

A page on another site POSTs to /todos on its own. The server can't tell, from the request alone, which page sent it — and the starter switched the check off for /todos. A to-do you never typed appears.

WHY THE FIX WORKS

@csrf puts a per-session token in the form. The attacker's page can't read it (the browser keeps it from other origins), so it can't copy it. Laravel checks the token on every POST — unless you switch the check off. Two layers you get free in 2026: the session cookie is SameSite=Lax, and Laravel 13 reads the browser's Sec-Fetch-Site header. The token is the layer that works in every browser.

THE FIX, IN ONE LINE

@csrf in the form; delete the except line

CSRF, in two lines

A01 · REQUESTS · DRILL 3

The attack

Another site makes your browser send a request your server treats as yours. From the request alone, the server cannot tell which page sent it.

A01 · TOKEN

The defense

Three layers: a SameSite cookie, the browser's Sec-Fetch-Site header, and a token only your own pages can read. @csrf in Blade; the XSRF-TOKEN header in your React api.js.

The idea under all of them: a cookie proves whose browser it is, not who meant to send the request. The token proves intent, in every browser.

The platform

A02 and A03:2025. Not a line of your code — the secrets you store, the packages you install, and the headers you send (or don't).

Drills 4, 5 and 6.

2. Security Misconfiguration

The code is fine; the way the app, server or platform is set up is not.

WHAT IT IS

Debug mode left on, default accounts, verbose error pages, unnecessary services, missing security headers, secrets in the repository.

Rose to 2nd in 2025 because every layer has settings and the safe value is rarely the default.

Nobody "writes" this bug. It is what is true when nobody checked.

EXAMPLES

`APP_DEBUG=true` in production: a crash shows the stack trace, file paths and environment values to the visitor.

A storage bucket set to public; an admin panel at its default URL with the default password.

No X-Frame-Options header, so your page can be framed inside a site that captures the clicks (drill 6).

`.env` committed to git with a live API key in it (drill 4).

HOW LARAVEL HELPS

`.env + .gitignore` from the first install: settings live outside the repository; config/ reads them once.

`APP_DEBUG=false` and APP_ENV=production: errors become a plain page, details go to the log.

`config:cache` in production, so env() outside config/ returns null and the mistake shows up early.

Headers are not a default: your SecurityHeaders middleware adds the four (Homework 8).

3. Software Supply Chain Failures

Harm arrives through code you did not write: a package, a build tool, a base image, a CI step.

WHAT IT IS

Your app is a few hundred lines of yours on top of a few hundred packages from strangers, pulled by composer and npm.

A dependency with a known hole, a malicious update, a stolen publishing token, a build server that was tampered with.

New in 2025, and 2nd in the community survey: the attacks of 2024–25 were mostly here.

EXAMPLES

Shai-Hulud (npm, Sept 2025): a worm that published itself into packages using maintainers' stolen tokens.

XZ Utils (2024): two years of social engineering to become a maintainer, then a backdoor in a compression library.

`postinstall` script in a package that reads your home directory when you run npm install.

`symfony/mime` in the pantry example's lockfile had a published advisory; composer audit found it (drill 5).

HOW LARAVEL HELPS

`composer.lock` pins the exact version of every package, so a clone installs what you tested, not whatever is newest.

`composer audit` checks that lockfile against the public advisory database; npm audit does the same for web/.

Laravel itself ships security releases on a published schedule; stay on a supported major.

Not automatic: someone runs the audit. Homework 8 makes it part of the process.

A secret committed in code

A02:2025 Security Misconfiguration *drill 4 · config/pantry.php, the notifications API key*

```
config/pantry.php   (committed)
'api_key' => 'sk_test_00000000000000000000000000',
'api_key' => env('PANTRY_API_KEY'),

.env   (listed in .gitignore, never committed)
PANTRY_API_KEY=sk_test_00000000000000000000000000

.env.example   (committed: the NAME, so teammates know to set it)
PANTRY_API_KEY=
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

The key sits in a file git tracks. Everyone with the repo has it — and deleting it next commit doesn't help: git log -p still shows it. A leaked key must be rotated, not just removed.

WHY THE FIX WORKS

.env is in .gitignore, so it never enters the repository. Code reads secrets with env(), in config/ files only. .env.example records the name so teammates know what to set. The key lives on each machine, not in git.

THE FIX, IN ONE LINE

env('PANTRY_API_KEY') + a line in .env

A dependency with a known hole

A03:2025 Software Supply Chain Failures *drill 5 · composer.json / composer.lock, the packages your app installs*

```
in a second terminal:
$ docker compose exec app composer audit

firebase/php-jwt  5.5.1
  Severity: critical
  Advisory: weak encryption / algorithm confusion

$ docker run --rm -v "$PWD":/app -w /app composer:2 \
    composer remove firebase/php-jwt --no-install --no-scripts
(in use? composer update firebase/php-jwt  moves past the advisory)
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

Your app installs a few hundred packages you never read. One of them, pulled in by one you chose, has a published, critical advisory. You didn't write it, but it ships in your app.

WHY THE FIX WORKS

composer audit (and npm audit) compare every installed version against a public advisory list. When one is flagged, you update past it or remove it. An install script is ordinary code that runs the moment you install.

THE FIX, IN ONE LINE

composer audit -> update or remove

Four headers, four things the browser refuses

A02:2025 Security Misconfiguration *drill 6 · php artisan make:middleware SecurityHeaders, one route, then bootstrap/app.php*

Header	Value	Tells the browser to refuse
`X-Content-Type-Options`	`nosniff`	guessing a file's type (MIME sniffing)
`X-Frame-Options`	`DENY`	showing your page in another site's frame (clickjacking)
`Content-Security-Policy`	`default-src 'self'`	loading scripts from anywhere but your origin
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	ever dropping back to plain HTTP (forces TLS)

```
php artisan make:middleware SecurityHeaders   (app/Http/Middleware/)
public function handle(Request $request, Closure $next): Response {
    $response = $next($request);
    $response->headers->set('X-Content-Type-Options', 'nosniff');
    $response->headers->set('X-Frame-Options', 'DENY');
    $response->headers->set('Content-Security-Policy', "default-src 'self'");
    $response->headers->set('Strict-Transport-Security',
        'max-age=31536000; includeSubDomains');
    return $response;
}
one route first: ->middleware(SecurityHeaders::class); then every route:
$middleware->append(\App\Http\Middleware\SecurityHeaders::class);
```

Why it works: a browser left without these headers guesses, and its guesses favour the attacker. Each header is one instruction, set once in a middleware that runs on every response. HSTS is how TLS stops being optional: after the first visit, the browser refuses http:// for your whole domain.

One pipe, three ways onto it

You wrote one. Every request already runs through several: each a handle() that refuses, changes, or passes on.



← the response comes back the same way, so an "after" middleware like SecurityHeaders sees it on the way out

GLOBAL · APPEND()

Every request, every response, a 404 too. Drill 6 step 4. For what is true of the whole app: headers.

GROUP · THE API AND WEB GROUPS

statefulApi() puts cookies, the session and the CSRF check on the api group for your SPA's domain. One line, three middleware.

ONE ROUTE · ->MIDDLEWARE()

Drill 6 step 3, drill 7. throttle:login on POST /login; auth:sanctum around the write routes. The rule sits next to the route it guards.

```
public function handle(Request $request, Closure $next): Response
{
before: look at $request; return early to refuse it (throttle, auth)
    $response = $next($request);    // the rest of the stack, then the controller
after: change $response on its way out (SecurityHeaders)
    return $response;
}
```

Six you have been using since the pantry example

Middleware	On the stack as	Refuses	Says	Where you met it
`EncryptCookies`	api group, via statefulApi()	a cookie someone edited: the signature no longer matches, so it is dropped	—	the session cookie, session 17
`StartSession`	api group, via statefulApi()	nothing — it loads who you are from the cookie, so auth can answer	—	login, week 12
`ValidateCsrfToken`	api group, via statefulApi()	a POST with no X-XSRF-TOKEN matching the session	419	drill 3 · HW 8
`throttle:login`	one route, ->middleware()	the 21st attempt in a minute for one email from one address	429	drill 7 · week 12 lab · HW 9
`auth:sanctum`	a route group	a request with no logged-in session	401	week 12 lab · HW 9
`SecurityHeaders`	global, append()	the browser's guesses: adds four headers on the way out	—	drill 6 · HW 8

The status code tells you who refused. 419, 429 and 401 come from the stack, before your controller runs. 403 (a policy) and 422 (validate()) come from inside it. When a request fails, the number says where to look.

throttle:login is one class with an argument

```
routes/api.php — the colon passes an argument to the class
Route::post('/login', [AuthController::class, 'login'])
    ->middleware('throttle:login');       // ThrottleRequests, limiter "login"
Route::middleware('auth:sanctum')->group(...); // Authenticate, guard "sanctum"

app/Providers/AppServiceProvider.php — what "login" means
RateLimiter::for('login', fn (Request $r) => Limit::perMinute(20)
    ->by(strtolower($r->input('email')).'|'.$r->ip()));

bootstrap/app.php — the order the stack is built in
$middleware->statefulApi();                // cookies, session, then CSRF
$middleware->append(SecurityHeaders::class); // last in, so its headers go on last
```

THE ARGUMENT

throttle:login and auth:sanctum are the same two classes every Laravel app ships; the word after the colon picks a limiter or a guard. Name a limiter once, in the provider, and reuse it on any route.

THE KEY

by(email | ip): twenty tries a minute per account per address. A password guesser is slowed on one account, and one address cannot try every account either. Return a 429, not a hint.

ORDER

Session before CSRF, because the token is checked against the session. auth before the policy, because the policy needs a user. SecurityHeaders appended last, so on the way out it runs last and stamps every response, errors included.

The platform, in three lines

A02 · SECRETS · DRILL 4

Secrets

Keys live in .env (git-ignored), read with env() in config/. A secret in git is leaked, even after you delete it.

A03 · DEPENDENCIES · DRILL 5

Supply chain

You ship code you didn't write. audit it against the advisory list; update or remove what's flagged.

A02 · HEADERS · DRILL 6

Headers & TLS

Send the four headers, and force HTTPS. The browser's defaults favour the attacker; override them.

The idea under all of them: none of these is your application code. Security is the whole system — what you store, what you install, what you send.

Six fixes, three design principles

Saltzer and Schroeder named these principles fifty years ago, before the web existed. Every one of today's fixes is one of them.

Complete mediation

Check every input, every time.

Binding and escaping check every value on the way in and out (drills 1, 2). A policy checked on every write, not once at login. An allow-list of fillable columns. validate() on every form.

Fail-safe defaults

Default to "no". Open on purpose.

CSRF on, headers set, debug off (drills 3, 6). The auth middleware answers 401 before the controller runs. The secure state is the one you don't have to remember.

Least privilege

Give each part only what it needs.

Secrets per machine, not in git (drill 4). A resident may edit their own items; only the RA deletes. A page that only hides buttons. A database user that can't drop tables.

You find each hole, then fix it

One to-do app, six holes, in inclass/wk11/laravel. Each drill shows you the attack first — run it, watch it work — then you make the one change that closes it, and press Check result. Same loop as weeks 7 and 9.

A05 · 8 MIN

1 · SQL injection

A search box that reads the users table. TodoController.

A05 · 6 MIN

2 · XSS

A note that runs in your browser. show.blade.php.

A01 · 7 MIN

3 · CSRF

Another site adds to-dos to your list. The form + bootstrap/app.php.

A02 · 5 MIN

4 · A leaked secret

An API key committed to git. config/pantry.php + .env.

A03 · 6 MIN

5 · A bad dependency

composer audit finds a critical advisory. composer remove.

A02 · 8 MIN

6 · Security headers

The four the browser needs. A middleware.

Build it before Friday (it downloads PHP, MySQL and the packages): `cd inclass/wk11/laravel && docker compose watch` then open http://localhost:8000.

This Friday, 11/6

DUE FRIDAY

Homework 7

Your team's Laravel back end: the table, the model, the seed data, and the five routes from your API.md. Bring it running — Homework 8 is the same six fixes, on it.

Plus: push your copy of Friday's lab to your repo at the end of class.

ASSIGNED FRIDAY

Homework 8 · team

Harden your api/: today's six, on your own project — a headers middleware, the CSRF and cookie settings, no raw SQL from input, no raw HTML in React, no secrets in the repo, the audit clean. Graded by reading your repo and requesting your API. Due Fri 11/13.

Homework 9 (Fri 11/13): the login and the write endpoints.

Readings for the lab (week 11 page, items 4 and 5): the Laravel Blade, Configuration and Sanctum sections, and four MDN security guides — the reference for drills 2, 3, 4 and 6, and for Homework 8.

Assume every input is hostile. Make every default fail closed.

That is the whole of today. The framework does it by default; your job is to notice where someone turned the default off — and on Friday, to turn it back on.

ITWS 2110 · WEEK 11 ·

The security lab, walked through

THE SHAPE OF THE LAB

Attack first, then fix. Insist on the order.

See the attack
run it; watch it work on your own app

→

Why it works
one sentence, from the drills page

→

The fix
one change, in the file the drill names

→

Check result
runs that drill's test on your files

→

Explain
to your partner, in a sentence

- A student who has watched /attack add a to-do to their own list understands CSRF in a way no definition delivers. The same for the UNION payload filling the list with emails. Don't let anyone read the fix first.
- Together on the projector: drills 1, 3 and 6 (the attack is visible; the fix is one line; the before/after is dramatic). Pairs: 2, 4, 5.
- The loop is the one from weeks 7 and 9: change one thing, save ("Syncing" in the watch terminal), press Check result. Two drills also need a second terminal (5) or a rebuild wait (4, 5).
- Hand-in at the end of class: copy the laravel folder into inclass/wk11/ in their own repo, commit, push — whatever they finished. Participation, not correctness.

Tuesday's thesis, to repeat on Friday: every one of these six is a framework default that the starter turned off. The job is to notice where, and turn it back on.

DRILL 1 · INPUT · 8 MIN · DO IT TOGETHER

SQL injection: a search box that reads the users table

THE ATTACK — RUN IT FIRST

On /todos, paste into the search box: ' UNION SELECT id, email, 0 FROM users WHERE email LIKE '% — the list fills with alex@example.edu and priya@example.edu as "titles". Read them aloud.

app/Http/Controllers/TodoController.php → index()

```
$q = trim((string) $request->query('q', ''));
~~$sql = "select id, title, done from todos~~
~~        where title like '%$q%' order by title";~~
~~$todos = collect(DB::select($sql));~~
$todos = Todo::where('title', 'like', "%$q%")->orderBy('title')->get();

// also accepted — still raw, but bound:
// DB::select('select … where title like ? order by title', ["%$q%"]);
```

Green is the answer. Red, struck through, is the starter line it replaces.

WHAT THE ROOM SEES

Before: every user's email, as to-do titles. After: the same payload gives "No to-dos match." — it is now a search for a title containing that whole string. ?q=rice still finds Buy rice.

WHY

Nothing about the value changed. What changed is where it travels: bound, the database receives the SQL and the value separately and can only compare the value, never run it. Week 3's prepare() and ?; every where() in week 9.

the check says, before `Drill1Test > a union payload cannot leak a user email — Expected: <!doctype html>… Not to contain: victim-secret@example.edu  (the ordinary-search test already passes)`

Pause: "what changed about the value?" Nothing. Trap: a fix that breaks ordinary search — the check also runs ?q=milk and expects Buy milk at store.

DRILL 2 · INPUT · 6 MIN

XSS: a note that runs in everyone's browser

THE ATTACK — RUN IT FIRST

Open /todos/3 (Pantry potluck). The seeded note is a <script> that alerts your cookies and what it can now do: read the page, post as you, send it all out — a stranger typed it into the shared list, and the page runs it.

resources/views/todos/show.blade.php

```
<h2>Notes</h2>
~~<div class="notes">{!! $todo->notes ?: 'No notes.' !!}</div>~~
<div class="notes">{{ $todo->notes ?: 'No notes.' }}</div>
```

Green is the answer. Red, struck through, is the starter line it replaces.

WHAT THE ROOM SEES

Before: an alert with whatever cookies JavaScript may read (Laravel's session cookie is HttpOnly, so it is not among them — worth pointing out). After: the note shows as text, angle brackets and all, and nothing runs.

WHY

{!! !!} prints the value as HTML, so a <script> becomes a real script. {{ }} escapes first: < becomes <, and the browser shows the characters. Blade escapes by default; the bug is the one place that asked it not to.

the check says, before `Drill2Test > a script in a note is escaped not executed — Expected: <!doctype html>… Not to contain: <script>alert('xss-marker')</script>`

Trap: deleting the notes line. The check also asserts the text still shows (assertSee alert('xss-marker')). Ask: why does Blade offer {!! !!} at all? Trusted HTML — and "trusted" is the whole question.

DRILL 3 · REQUESTS · 7 MIN · DO IT TOGETHER

CSRF: another site adds to-dos to your list

THE ATTACK — RUN IT FIRST

Open http://127.0.0.1:8000/attack — 127.0.0.1, NOT localhost: to the browser, another website. It POSTs to localhost:8000/todos on load; /todos then shows "Call the attacker", which nobody typed.

resources/views/todos/index.blade.php, then bootstrap/app.php

```
<form method="POST" action="{{ route('todos.store') }}" class="add">
    @csrf
    <input type="text" name="title" placeholder="New to-do" required>
</form>

// bootstrap/app.php — delete this line
~~$middleware->validateCsrfTokens(except: ['todos']);~~
```

Green is the answer. Red, struck through, is the starter line it replaces.

WHAT THE ROOM SEES

Before: the to-do appears. After both halves: reload 127.0.0.1:8000/attack — 419 Page Expired. Opened at localhost instead, nothing changes before or after: Laravel 13 lets a same-origin POST through on the browser's Sec-Fetch-Site header, token or not.

WHY

The server cannot tell from the request which page sent it. The token proves intent: your page can include it; the attacker's page cannot read it (other origin), so cannot copy it. 2026 footnote: SameSite=Lax already withholds the cookie on a cross-site POST — the token is the layer that doesn't depend on the browser.

the check says, before `Drill3Test > the add form includes a csrf token — To contain: name="_token"` · then: `bootstrap/app.php still excludes 'todos' from the CSRF check -- delete that line.`

Say it twice: this check reads files, not behaviour (next slide). Show the real 419 at 127.0.0.1. Trap: @csrf added, except line left in — the second test names it. Reseed afterwards.

DRILL 4 · REQUESTS · 5 MIN

A secret committed in code

THE ATTACK — RUN IT FIRST

Open config/pantry.php: the API key is written in a file git tracks. /settings shows the app using it. Ask: in a team of four, who has the key? Everyone — and git log -p keeps it after it is deleted.

config/pantry.php, then .env.example

```
// config/pantry.php
'api_key' => 'sk_test_000000000000000000000000',
'api_key' => env('PANTRY_API_KEY'),

// .env.example — lab shortcut: the image builds .env from this file
PANTRY_API_KEY=sk_test_000000000000000000000000
```

Green is the answer. Red, struck through, is the starter line it replaces.

WHAT THE ROOM SEES

/settings shows the same key before and after. That is the point: nothing about the running app changes, only where the secret lives.
Editing .env.example triggers a rebuild under watch — wait for it before Check.

WHY

.env is git-ignored, so the key lives on each machine, not in the repository; env() reads it, in config/ files only. Lab shortcut to say out loud: the Dockerfile copies .env.example to .env, so the lab puts the value in .env.example. In their project .env.example carries the name, no value.

the check says, before `Drill4Test > the key is not hardcoded in the config file — The API key is still written in config/pantry.php. Move it to .env and read it with env().` · then: Add PANTRY_API_KEY to .env.example…

Pause: "you deleted it in the next commit — are you safe?" No: rotate, don't just delete. Trap: adding the key to a local .env only — the second test wants it in .env.example.

DRILL 5 · THE PLATFORM · 6 MIN

A dependency with a published advisory

THE ATTACK — RUN IT FIRST

Second terminal: docker compose exec app composer audit → firebase/php-jwt 5.5.1, Severity: critical. Nobody here wrote it; it ships in the app. Ask: of ~80 packages, how many has anyone read?

composer.json / composer.lock — on the host, so the change is in the files you push

```
# audit runs in the app container (same lockfile as the image):
docker compose exec app composer audit

# remove it in a throwaway Composer container that shares the folder,
# so composer.json and composer.lock change on YOUR machine:
docker run --rm -v "$PWD":/app -w /app composer:2 \
  composer remove firebase/php-jwt --no-install --no-scripts

# watch sees composer.lock change and rebuilds. Then: audit again -> clean.
```

Green is the answer. Red, struck through, is the starter line it replaces.

WHAT THE ROOM SEES

The watch terminal prints Rebuilding; wait for it. /settings flips from installed to not installed; composer audit says no advisories. The lockfile in their repo no longer pins php-jwt.

WHY

audit compares every locked version against a public advisory list. Flagged: update past it or, if unused (it is), remove it. The host-side command matters: composer remove inside the running container only changes the container's copy, which the image rebuild and their push would never see.

the check says, before `Drill5Test > no vulnerable php jwt in the lockfile — composer.lock still pins firebase/php-jwt 5.5.1, which has a published advisory. Run: … composer remove firebase/php-jwt`

Trap: pressing Check before Rebuilding finishes. Second trap: `exec app composer remove` passes the Check in that container but leaves the pushed lockfile vulnerable — see the notes.

DRILL 6 · THE PLATFORM · 8 MIN · DO IT TOGETHER

Security headers: four things the browser refuses

THE ATTACK — RUN IT FIRST

DevTools → Network, reload any page, click the request, read Response Headers: none of the four. The browser is guessing — it will frame the page, sniff types, run scripts from anywhere.

make:middleware, one route in routes/web.php, then bootstrap/app.php

```
$response = $next($request);
$response->headers->set('X-Content-Type-Options', 'nosniff');
$response->headers->set('X-Frame-Options', 'DENY');
$response->headers->set('Content-Security-Policy', "default-src 'self'");
$response->headers->set('Strict-Transport-Security',
    'max-age=31536000; includeSubDomains');
return $response;
// step 3: ->middleware(...) on /todos only. Step 4, every route:
$middleware->append(\App\Http\Middleware\SecurityHeaders::class);
```

Green is the answer. Red, struck through, is the starter line it replaces.

WHAT THE ROOM SEES

Reload with Network open: all four under Response Headers, on every page — the drills page too, since the middleware is appended globally. Map each to its MDN guide from the reading.

WHY

Each header is one instruction to the browser: don't sniff, don't frame, scripts only from me, never plain HTTP again. A middleware sets them once, for every response. HSTS is the TLS tie-in: TLS encrypts the connection, HSTS makes the browser insist on it.

the check says, before `Drill6Test > every response carries the four security headers — No X-Content-Type-Options: nosniff header on /todos. (After step 3: … on /. /todos has it, so the middleware is on one route only.)`

Trap: the middleware is written but not appended — same message. Ask one pair per header: the attack it stops, in a sentence. (Clickjacking is the one to draw.)

THE THING TO EXPLAIN TWICE

Three checks read your files, not your app's behaviour

Drill	What the check does	Why not a request?
1 · SQL	GETs /todos?q=<payload>; asserts no email in the page; GETs ?q=milk; asserts the match	it can — the attack either works or it doesn't
2 · XSS	GETs /todos/{id}; asserts the raw <script> is absent and the escaped text present	it can
3 · CSRF	reads the form for name="_token"; reads bootstrap/app.php for the except list	Laravel's test runner skips the live CSRF middleware, so a PHPUnit request can't tell protected from unprotected. Show the 419 in the browser instead.
4 · secret	reads config/pantry.php (no literal key; an env() call); reads .env.example for the name	a secret is dangerous because it is in git — a fact about the repository
5 · dependency	parses composer.lock; asserts php-jwt absent or ≥ 6.0	a vulnerable dependency is a fact about the lockfile
6 · headers	GETs /; asserts the four headers	it can

That contrast is itself the lesson: some security properties are about what the system does, and some are about what is written down.

RUN OF SHOW · FRI 11/6

Where each drill goes, and what to say when

Minutes	Drills	Mode	Stop the room for
0–10	setup recap	all	port 8000 (stop week 9 / HW7 first); everyone at localhost:8000 with drill 1 open; the attack-first rule
10–18	1	together	the payload; the emails; one line changed; the same payload again. "What changed about the value?"
18–30	2, then 4	pairs	2: why {!! !!} exists · 4: who has the key once it is committed; wait for the rebuild
30–38	3	together	/attack adds a to-do; "the cookie was valid — so why isn't this legitimate?"; the 419 after
38–50	5, then 6	pairs	5: start it first (rebuild wait), host-side command · 6: Network tab before and after
50–58	6 debrief	together	the four headers on the projector; one pair per header names the attack it stops
58–60	hand-in	all	push inclass/wk11/laravel to their repo; HW7 due tonight; HW8 (harden your api/: the same six) assigned, due 11/13

Exit prompt: name one of today's six the framework had already fixed for you before you ever saw it — and say what the starter did to un-fix it.

WHEN SOMETHING BREAKS

The table from the README, plus two from the dry run

What they see	First thing to check
The first docker compose watch takes minutes	Normal, once: PHP, MySQL and the packages are downloading
port is already allocated	Week 9 or HW7 is on 8000. Stop it, or APP_PORT=8001 docker compose watch
Drill 1 still fails after the edit	The check also runs an ordinary search; ?q=rice must still list Buy rice
Drill 2: the note is gone, not escaped	They deleted it. Keep {{ $todo->notes }} so the text still shows
Drill 3 passes the form check but not the route check	@csrf added, except line still in bootstrap/app.php
Drill 4 fails on .env.example	PANTRY_API_KEY must be in .env.example, not only .env; wait for the rebuild it triggers
Drill 5 still fails after composer remove	Wait for Rebuilding to finish. If they used exec app composer remove: host lockfile unchanged — use the docker run … composer:2 form
Drill 6: headers not showing	Middleware written but not appended in bootstrap/app.php
419 Page Expired on the add form after drill 3	Correct on a stale tab: reload /todos for a fresh token, then add

Don't let anyone edit the tests to get a PASS. The checks are the same files docker compose run --rm test runs.

The lab, in three moves

1 · ATTACK FIRST

See the failure

Run the exploit before showing the answer. Make the consequence visible on the app students are about to fix.

2 · RESTORE THE DEFAULT

Fix one thing

Bind or escape input; restore CSRF; move secrets; audit packages; add headers. Change only the named file.

3 · PROVE IT

Check and explain

Press Check result, confirm the before/after, then explain in one sentence why the fix closes the hole.

The idea under all six: security is a property of the whole system. Assume input is hostile, fail closed, and notice when a safe framework default was turned off.